Formalisme technique et opérationnel du modèle AAM O94.2

Dossier de conception et de vérification AAM

19 octobre 2026

Table des matières

1 Objet, portée et principe de preuve **1**

2 Définitions des briques de base **2**
2.1 Compétence, méthode, règle . . . 2
2.2 Composition exécutable . . . 2

3 Observation boîte noire : modes et pipeline **2**
3.1 Mode A — dataset ou lot de cas . . . 2
3.2 Mode B — cible persistante, streaming en direct . . . 3
3.3 Moteur de sondes et diversité . . . 3
3.4 Apprentissage de capsule et statut de reproduction . . . 3

4 Intégration des modèles externes et locaux **4**
4.1 Adaptateurs disponibles . . . 4
4.2 Politiques d'entraînement . . . 4

5 Interface et expérience utilisateur **4**

6 Architecture et pipelines de bout en bout **5**
6.1 Observation live . . . 5
6.2 Chargement et intégration d'un modèle local . . . 5
6.3 Programmation par composition . . . 5

7 Commandes opérationnelles **5**

8 Conséquences **6**
8.1 Théoriques . . . 6
8.2 Conceptuelles . . . 6
8.3 Opérationnelles . . . 6

9 Statut de conformité et limites ouvertes **6**

1 Objet, portée et principe de preuve

O94.2 est une incrémentation de O94.1. Elle traite trois défauts opérationnels signalés lors des essais : (i) l'ingestion de traces était trop rigide et son plafond de cas n'était pas exposé correctement, (ii) l'observation en action ne proposait pas une boucle de sondes envoyées à une cible persistante, et (iii) l'intégration de modèles locaux et la programmation par composition étaient insuffisamment visibles dans l'interface.

Le présent document spécifie les contrats de l'incrément. Il ne transforme pas une réussite de test sur une cible simulée en preuve universelle de clonage d'un programme arbitraire. Une reproduction est qualifiée par la nature de ses observations, la portée des entrées observées, les budgets et les

tests hors apprentissage. Les échecs d'exécution et les limites environnementales restent enregistrés ; le potentiel général du modèle n'est pas déclaré d'avance.

2 Définitions des briques de base

2.1 Compétence, méthode, règle

— **Compétence de base** : opération générale ayant un contrat d'entrée/sortie explicite, exécutable et observable (par exemple observer, créer une empreinte, sélectionner une compétence, vérifier un critère, enregistrer une relation causale).

— **Méthode de base** : politique de transformation ou d'enchaînement réutilisable entre compétences (par exemple composition séquentielle, observation multi-vues, recherche adaptative de sondes, résolution itérative).

— **Règle de base** : contrainte déclarative portant sur l'accès, la provenance, la validation, l'arrêt, la séparation train/holdout ou l'évaluation d'une étape.

Ces classes sont complémentaires : une méthode n'est pas une compétence simplement parce qu'elle s'exécute, et une règle n'est pas une réponse. Chaque bloc possède un identifiant, une description, des préconditions, des dépendances et une référence d'implémentation. Le catalogue des briques (fichier `aam_o93/base_blocks.py`) constitue la base déclarative. Les sous-systèmes historiques peuvent rester encapsulés par adaptateurs ; la métadonnée de composition ne prétend pas que chaque ligne du runtime a été réécrite à partir de blocs atomiques.

2.2 Composition exécutable

L'éditeur O94.2 utilise un DSL JSON versionné `aam-o94-2-program-composition-v1`. Les nœuds admis sont compétence, méthode séquentielle et règle conditionnelle. Le compilateur résout tous les identifiants contre les catalogues courants avant de produire le graphe de compétences. Aucun `eval`, `exec` ou texte Python arbitraire n'est accepté comme définition de composition. Les paramètres sont des données structurées et restent soumis au contrat de la compétence ciblée.

Exemple schématique :

```
{
  "schema": "aam-o94-2-program-composition-v1",
  "root": {
    "kind": "method", "method_id": "M-SEQUENCE",
    "steps": [
      {"kind": "competence", "competence_id": "observe"},
      {"kind": "competence", "competence_id": "create_fingerprint"}
    ]
  }
}
```

L'identifiant d'une compétence doit exister dans le registre AAM. Les compositions sauvegardées conservent l'origine des nœuds (type de brique et identifiant de base) pour l'audit. Une compilation valide est une preuve structurelle ; elle ne prouve pas automatiquement l'équivalence fonctionnelle de la composition pour toutes les entrées.

3 Observation boîte noire : modes et pipeline

3.1 Mode A — dataset ou lot de cas

Le chargeur supporte JSON, JSONL/NDJSON, CSV et TSV/TAB. La racine JSON peut être une liste, un objet unique ressemblant à un enregistrement, un objet indexé par identifiant, ou un objet contenant une collection dans `cases`, `case`, `data`, `records`, `rows`, `items`, `examples`, `samples`,

`observations` ou `events`. La clé de collection peut également être indiquée manuellement. Le mode déduit par défaut les champs entrée (`input`, `prompt`, `question`, `source`, `request`, `x`, `text`, `message`, `instruction`, `content`, etc.), cible/attendu (`expected`, `target`, `label`, `answer`, `response`, `output`, `result`, `y`) et partition (`split`, `partition`, `subset`, `dataset_split`, `fold`, `data_split`, `set`). Un chemin imbriqué tel que `payload.text` peut être forcé.

La valeur par défaut de l'UI est 2 000 cas, réglable entre 1 et 20 000 ; le garde-fou dur du chargeur est 20 000 cas et la taille brute maximale du fichier d'entrée est 256 MiB. Le nombre total de secondes, le délai par cas et la sortie maximale sont des budgets séparés. Si les données dépassent le budget choisi, AAM fournit une erreur indiquant quel paramètre adapter au lieu d'une erreur générique « attendu cases ».

Le champ attendu est facultatif lors d'une acquisition boîte noire : il sert à une comparaison indépendante s'il existe et n'est pas fourni au mécanisme de découverte des sondes. Les chemins de fichier d'origine et les hashes sont conservés dans les reçus ; le contenu fourni est traité comme donnée non fiable.

3.2 Mode B — cible persistante, streaming en direct

`BlackboxStreamSession` démarre, après confirmation explicite, une cible en processus persistant, à l'aide d'une liste d'arguments `argv` et `shell=False`. Contrat minimal de l'adaptateur : la cible lit une valeur JSON par ligne depuis `stdin` et émet une réponse par ligne vers `stdout`. `stderr` est séparé et n'est jamais interprété comme la sortie fonctionnelle. Chaque échange reçu est ajouté immédiatement, avec synchronisation disque, à une trace JSONL ; les événements de cycle de vie sont dans un fichier latéral `.events.jsonl`.

Les budgets configurables sont : temps total (0,1–3600 s), nombre maximal d'échanges (1–20 000), délai de réponse (0,1–300 s), et octets maximaux par réponse (1 KiB–2 MiB). La fermeture du runtime arrête les cibles encore actives. Les processus enfant sont exécutés avec les privilèges du compte courant : l'utilisateur doit donc n'autoriser que des programmes qu'il contrôle ou a la permission de tester.

3.3 Moteur de sondes et diversité

Le bouton *Générer et envoyer des sondes* fabrique à partir d'une valeur JSON graine un ensemble de variantes distinctes : valeurs numériques voisines, chaînes transformées, permutations/variantes de listes et mutations locales de champs de dictionnaires. Les sondes déjà observées dans la session sont exclues de la ronde suivante. Chaque sonde passe réellement par la cible, et la réponse enregistrée est l'observation, non une valeur attendue codée en dur.

Ces mutations constituent une politique de sondage générique ; elles ne garantissent pas que chaque entrée respecte le schéma sémantique de toute cible. Les erreurs et refus éventuels de la cible sont des observations à conserver. Pour couvrir une API métier, une GUI, une base de données, un OCR, une caméra ou un équipement, un adaptateur dédié doit exposer ses entrées/sorties observables et être autorisé ; il ne faut pas confondre le protocole ligne JSON avec l'attachement universel aux états internes d'un objet opaque.

3.4 Apprentissage de capsule et statut de reproduction

Les traces d'échanges ont la forme `{input,target,split,...}` et sont ré-ingérables par l'outil de capsule. La séparation train/holdout est conservée. La capsule hors source actuelle fournit une reproduction exacte des entrées rencontrées et explicitement enregistrées ; une entrée inconnue reste `UNRESOLVED`. Le taux sur holdout mesure uniquement le holdout réellement défini. Le terme *clonage fonctionnel général* n'est autorisé qu'avec des expériences supplémentaires de comportement, de couverture, de tests indépendants et de critères explicites.

4 Intégration des modèles externes et locaux

4.1 Adaptateurs disponibles

L'espace de travail « Modèles locaux / PyTorch » expose :

1. `transformers_local` : répertoire de modèle local compatible Transformers, poids/tokenizer/config présents, `local_files_only=True` et `trust_remote_code=False` ; PyTorch est utilisé par l'implémentation de génération lorsque ses dépendances sont installées.
2. `pytorch_adapter` : adaptateur Python local dont le propriétaire connaît l'architecture du checkpoint brut. Le fichier est hashé et doit être autorisé par une case à cocher explicite avant son exécution. Contrat requis : `load_model(context)` et `generate(model,prompt,options)` ; contrat d'entraînement optionnel : `train_model(context,model,train_rows,output_dir,policy)`. Un fichier `.pt` ou `.pth` seul ne révèle pas de façon universelle une architecture ou un tokenizer.
3. `ollama` et `openai_compatible` : endpoints configurés explicitement. L'accès distant dépend de l'activation Internet, d'une allowlist et de HTTPS hors boucle locale ; les secrets sont lus depuis l'environnement et ne sont pas enregistrés dans le registre.
4. Backend AAM pair : l'utilisateur choisit un workspace AAM déjà initialisé (par exemple avec `o93_settings.json`) qui est exposé au sélecteur de chat. Une archive du modèle doit d'abord être extraite et démarrée pour produire son propre workspace ; une archive zip n'est pas un backend vivant.

4.2 Politiques d'entraînement

`inference_only` ne met pas à jour les poids. `export_supervised_pairs` prépare les paires d'apprentissage et ne modifie pas les poids. `transformers_lora` entraîne un adaptateur LoRA uniquement lorsque dépendances, format des poids et modules cibles conviennent. `adapter_train` délègue le mécanisme d'entraînement au `train_model` de l'adaptateur explicitement autorisé ; AAM fournit uniquement les lignes train, et exclut les lignes holdout. L'adaptateur est responsable du chargement des poids, de leur sauvegarde et de la définition d'une évaluation ; le reçu AAM ne prétend pas qu'une mise à jour s'est produite si l'adaptateur ne la rapporte pas clairement.

Après intégration, l'orchestration AAM peut combiner les sorties du modèle avec compétences, méthodes, règles, RAG et mémoire. Cela n'implique pas que tous les modèles multimodaux ni chaque politique d'entraînement soient interchangeables : chaque adaptateur doit documenter les modalités, les entrées, les sorties, le budget mémoire et les tests qu'il supporte.

5 Interface et expérience utilisateur

Les onglets dédiés sont :

— **Observer** : observation fichier/dossier, lot tabulaire avec mapping des clés et budget, capture active d'un lot, session streaming persistante, sondes manuelles et automatiques, clôture/trace, accès caméra soumis à autorisation.

— **Modèles locaux / PyTorch** : enregistrement, liste, test de génération, dossier Transformers, adaptateur PyTorch explicite, checkpoint, choix de politique train, export des données ou entraînement par adaptateur, connexion à un workspace AAM pair.

— **Programmation par composition** : DSL JSON, validation, sauvegarde et exécution de compositions sûres, ainsi que liste des programmes persistés.

Les anciens panneaux restent en place pour conserver la rétrocompatibilité. Les catalogues de composition se chargent à la demande pour éviter de concurrencer l'initialisation du statut et de l'autonomie au lancement de la fenêtre.

6 Architecture et pipelines de bout en bout

6.1 Observation live

1. l'utilisateur choisit la commande cible, le répertoire de travail et un budget explicite ;
2. l'UI demande une autorisation visible ;
3. le runtime démarre le processus persistant sans shell et ouvre les canaux ;
4. les sondes sont saisies ou générées à partir d'une graine ;
5. chaque paire réellement reçue est hashée, horodatée, ajoutée durablement au JSONL et journalisée dans la mémoire causale ;
6. l'utilisateur ferme la session ou le budget provoque un arrêt borné ; le rapport résume durée, échanges, sorties, motif d'arrêt et hashes ;
7. une capsule peut ensuite être construite, séparée des sources, et évaluée sur les cas de test effectivement disponibles.

6.2 Chargement et intégration d'un modèle local

1. sélectionner le backend, le modèle ou checkpoint, son adaptateur éventuel et la politique ;
2. valider chemin, format, consentement local et contraintes réseau ; enregistrer un profil versionné avec hash de l'adaptateur ;
3. invoquer un test de génération distinct de l'enregistrement ;
4. si l'entraînement est demandé, normaliser entrée/cible, séparer le holdout, exécuter seulement la politique déclarée ;
5. archiver reçu, dataset exporté ou artefact et rapport d'évaluation. Une préparation de données ne reçoit jamais le statut d'entraînement réel.

6.3 Programmation par composition

1. sélectionner des compétences existantes et déclarer la séquence/la règle ;
2. compiler le DSL contre le catalogue de briques, valider les contrats et garder la provenance ;
3. persister sous une identité de programme non ambiguë ;
4. exécuter sur une entrée explicite et conserver résultats/erreurs ;
5. promouvoir une composition en compétence réutilisable seulement après critères de validation indépendants.

7 Commandes opérationnelles

Depuis le dossier extrait :

```
# Windows
run_AAM_O94_2.bat

# macOS / Linux
./run_AAM_O94_2.sh

# Diagnostic sans ouvrir l'UI
python -m aam_o93 preflight

# Tests ciblés
python -m pytest -q tests/test_o942_live_observation.py
python -m pytest -q tests/test_o941_model_integration.py
```

Dans le panneau de capture tabulaire, configurer `Maximum de cas`, les clés entrée/cible/split et, lorsque nécessaire, `Clé collection JSON`. Dans le panneau live, configurer les quatre budgets ; arrêter proprement la session avant de traiter sa trace.

8 Conséquences

8.1 Théoriques

L'observation est formalisée comme échantillonnage actif de comportements observables, et non comme supposition des états internes. Les sondes doivent varier l'entrée de manière traçable ; les résultats guident des hypothèses, alors que le holdout indépendant teste la portée réelle de la reproduction. La composition articule les briques dans un graphe versionné plutôt que de masquer la programmation dans un bloc arbitraire.

8.2 Conceptuelles

La séparation compétence/méthode/règle est explicite. Un modèle de langage devient un fournisseur de compétence parmi d'autres, tandis que RAG, mémoire, orchestration, observation et vérification forment un cadre autour de lui. Les adapters conservent les particularités de chargement et d'entraînement propres à l'architecture.

8.3 Opérationnelles

L'UI expose les budgets et les clés au lieu d'imposer une valeur silencieuse. Les erreurs sont plus diagnostiques ; les traces live sont produites pendant l'échange. L'exécution de code adaptateur local exige un geste explicite et est limitée au chemin choisi, mais elle demeure une exécution de code local potentiellement dangereuse si la source n'est pas de confiance.

9 Statut de conformité et limites ouvertes

Les tests locaux vérifient les schémas 1484 lignes sous `case`, les alias de colonnes, les clés imbriquées, le CSV, les budgets, le streaming d'une cible persistante, la trace ré-ingérable, la génération de sondes distinctes, l'intégration d'un adaptateur simulé et la compilation de DSL sûr. Le test UI du nouvel onglet doit être exécuté sous un serveur graphique ; un exécutable Windows natif, des poids externes réels, l'entraînement réel du checkpoint fourni par l'utilisateur, la capture de GUI arbitraire et les dispositifs physiques ne peuvent pas être réputés testés sans leurs environnements réels.

Les écarts restants (capture de modalités par adaptateurs dédiés, équivalence logicielle générale, robustesse de chaque politique d'entraînement et évaluations avec poids réels) restent ouverts dans les registres d'erreurs. Aucun écart ouvert n'est interprété comme une conclusion de principe sur le potentiel futur d'AAM.